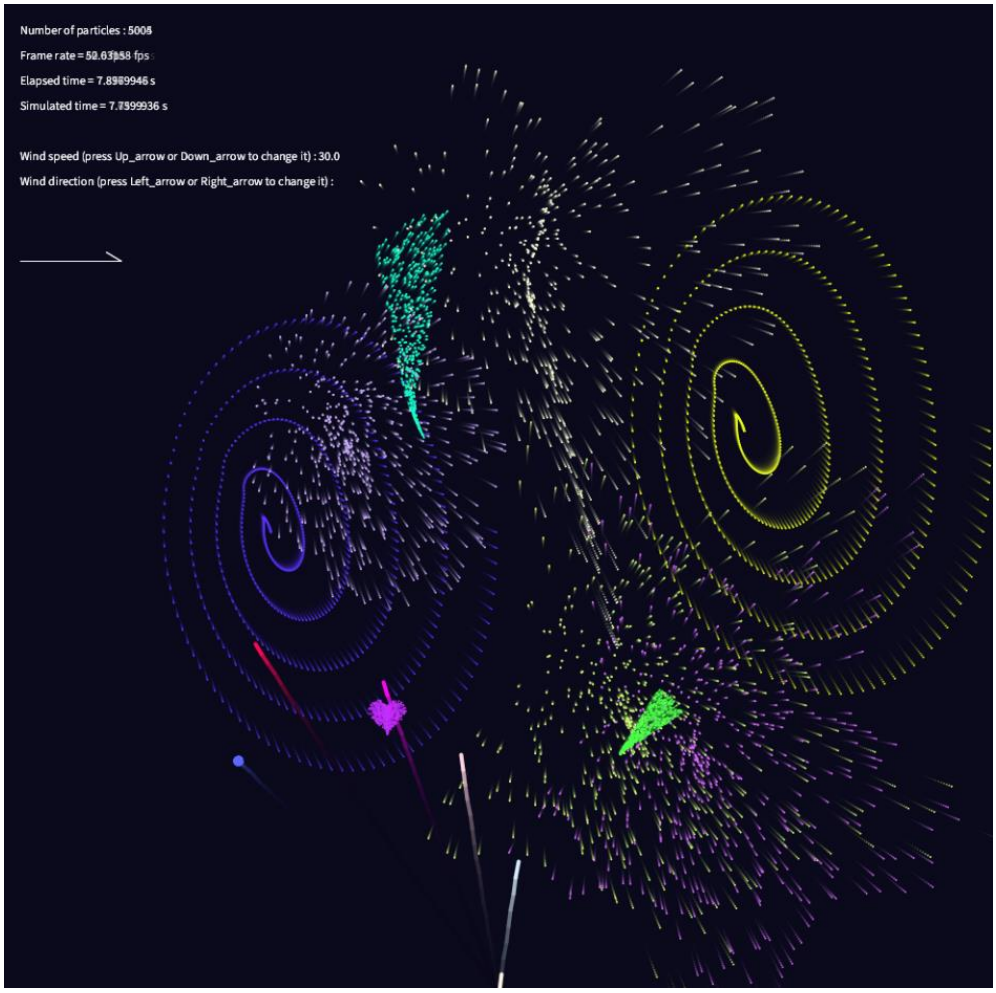
## Introducción

Durante esta práctica se ha pretendido trabajar el desarrollo de sistemas de partículas a través de la simulación de un castillo de fuegos artificiales. De forma que al presionar en la pantalla se lanza un proyectil en esa dirección que explota tras un tiempo de vida, liberando un determinado número de partículas extra definidas en código, constando todas ellas de un tiempo de vida también, y viéndose afectada esta clase Partícula que las define por la fuerza de la gravedad y el viento.

Cada proyectil, de forma aleatoria, producirá en la explosión una palmera diferente entre las diferentes opciones programadas.

En este documento se detallará ciertos elementos del desarrollo y se estudiará el impacto computacional del número de partículas en pantalla.




## Detalles del computador de pruebas

Para las pruebas se ha utilizado un computador MSI GL65 95EK con los siguientes detalles:

- Configuración mediante el driver de NVIDIA para el uso específico de la tarjeta gráfica dedicada: NVIDIA RTX 2060. Se ha comprobado a través del administrador de tareas que la carga de dibujado que supone la simulación aparece reflejada únicamente en dicho procesador.
- Uso de Windows 10, lo que supone la máxima compatibilidad con el SW de NVIDIA.

- Simulación realizada en el monitor secundario conectado por HDMI (por pruebas realizadas anteriormente, a veces implica una reducción del rendimiento por el acceso a la zona de memoria del mismo).

**Fuerzas, constantes y coeficientes**

Las fuerzas que actúan constantemente durante la simulación son dos:

1. **La fuerza de la gravedad.** Generada a través de la constante de aceleración terrestre pero que, a diferencia de la práctica anterior, pues en este caso no convertimos las coordenadas de pantalla a coordenadas de mundo, esta vez actúan en el sentido positivo del eje Y, que coincide con la dirección de bajada.

2. **La fuerza del viento.** El sistema de viento elegido para producirla consiste en lo siguiente:

    a. En el documento de la práctica se indica que cuando la velocidad de la partícula tiene sentido contrario al viento, se le ejerce una fuerza proporcional a la velocidad de la partícula y la del viento sumadas. Además, se indica que la partícula no será empujada por el viento si el sentido de su velocidad es el mismo y la velocidad de la partícula es mayor o igual que la de este.

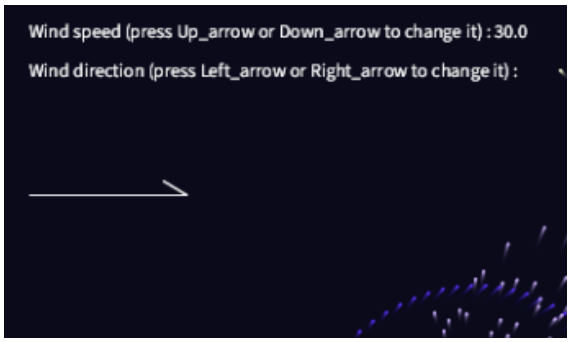
    b. En un principio, a través del signo del producto escalar, se ha planteado establecer una condición que siga las instrucciones anteriormente mencionadas; sin embargo, los resultados no eran adecuados: la diferencia entre la injerencia del viento en las partículas según su sentido era demasiado grande y las palmeras parecían cortarse por la mitad.

    c. En consecuencia, se decide utilizar el producto escalar de ambas velocidades para dar lugar a un coeficiente de 0 a 1 inversamente proporcional a la alineación de sus vectores; de forma que se cumplen ambas condiciones del apartado 'a' y el viento actúa en menor proporción conforme más alineado está con la velocidad de la partícula.

```
PVector Fw = _windVelocity.copy().normalize();
float vDot = PVector.dot(Fw,_v.copy().normalize()); //[-1,1]
float KA = 1 - ((vDot+1.0)*0.5);                    //[0,1] propor. inversa a orietacion con el viento
float v_mag = _v.mag();
Fw.mult(WIND_CONSTANT*KA*(v_mag+WIND_SPEED));
```

    d. La fuerza ejercida por el viento se calcula como la dirección de la velocidad del viento multiplicada por el anterior coeficiente (KA), la suma de los módulos de la velocidad del viento y de la partícula y una constante de transformación de velocidad en fuerza aparente (WIND_CONSTANT).

    e. La velocidad y dirección del viento son personalizables a través de la interfaz, mientras que la última constante mencionada está programáticamente fijada a 0.2.



**Tipos de palmeras**

Para definir las diferentes palmeras se han utilizado en todos los casos ecuaciones circulares o polares; es decir, ecuaciones cuyo valor de entrada se trataría de un ángulo y que por tanto definen, para cada valor angular, la dirección de la velocidad inicial de la partícula correspondiente a ese ángulo de salida. Las magnitudes de dichas velocidades son calculadas a parte a través de una proporción aleatoria de una velocidad de partícula regular establecida por defecto en el programa.

Además, los ángulos de entrada se definen según una proporción de 2*PI rad (es decir, de una vuelta) que depende del número de partículas generadas.

Los procesos específicos seguidos para cada palmera definida son los siguientes:

- **Palmera circular.** La velocidad viene definida exclusivamente por el seno y coseno del ángulo, con los que se puede definir un vector unitario perfecto para la dirección.

- **Palmera de corazón**. En este caso, utilizamos el siguiente sistema multiplicado por un facto de escala de 0,3 y negando v_y por lo que indicamos anteriormente de la dirección de abajo.

  $$v_x = 3\sin\alpha - \sin 3\alpha \qquad v_y = 3.25\cos\alpha - 1.25\cos 2\alpha - 0.5\cos 3\alpha - 0.25\cos 4\alpha$$

- **Palmera de estrella.** El proceso de este caso es aún más complejo y viene definido por la siguiente ecuación:

  $$r = r_0 + \frac{1}{c}\sqrt{-\log[\,2e^{-a^2} - e^{-b^2 x_{max}^2 \sin^2\{(\phi - \frac{\pi}{2})\cdot\frac{n}{2}\}}\,]}$$

  a=0.81
  r0=0.1*xmax,
  b=0.14
  c=1

  Dicho resultado r se aplica como un factor de escala extra para el vector unitario del vector definido de la misma forma que en el proceso del círculo.

- **Palmera progresiva.** Esta palmera sigue nuevamente el proceso del círculo, pero en este caso el ángulo de entrada se multiplica por 5 y el vector velocidad se multiplica por un valor descendente hasta 0 para producir un efecto de espiral.

- **Palmera cañón.** Si bien se mantiene el proceso de producción de velocidades utilizado en el caso del círculo, en este caso el ángulo de entrada solo puede tener valores colindantes al ángulo de la última dirección que había tomado la carcasa, en concreto valores entre del rango

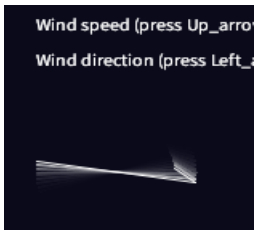
  $[\alpha_{carcasa}, \alpha_{carcasa} + 0.5]\ \ \alpha_{carcasa} > \frac{PI}{2} + 0.1$

  $[\alpha_{carcasa} - 0.5, \alpha_{carcasa}]\ \ \alpha_{carcasa} < \frac{PI}{2} - 0.1$

  $[\alpha_{carcasa} - 0.25, \alpha_{carcasa} + 0.25]\ \ \alpha_{carcasa} \epsilon\ [\frac{PI}{2} - 0.1, \frac{PI}{2} + 0.1]$

  Esto permite aparentemente orientar la dirección de salida del cañón de partículas de forma más visual que indicando un único rango para cualquier valor del ángulo de la carcasa.

## Mejoras visuales de la simulación




Más allá de elegir unos valores adecuados de tamaño de partícula, se ha recurrido a las siguientes herramientas para dar una buena apariencia a la simulación:

1. Para mostrar de forma intuitiva la dirección del viento se dibuja una flecha con punta que rota conforme el usuario pulsa las teclas adecuadas.
2. Para el redibujado de la escena, se recurre en vez de a la función background(), a la creación de un rectángulo con un cierto grado de transparencia que permite generar una estela adecuada para todas las partículas sin a penas coste adicional.

```
//background(BACKGROUND_COLOR[0], BACKGROUND_COLOR[1], BACKGROUND_COLOR[2]);
fill(BACKGROUND_COLOR[0], BACKGROUND_COLOR[1], BACKGROUND_COLOR[2],60.0);
rect(-10,-10,width+10,height+10);
```

3. Los colores de los fuegos artificiales son definidos en HSB para permitir establecer un brillo máximo por defecto y evitar generar colores aleatorios oscuros.
4. El tamaño de dibujado de la partícula de carcasa es el doble que el de las partículas de explosión. Además, la carcasa no es un círculo, sino una línea de magnitud del paso de simulación con un valor de grosor (contorno) igual al tamaño definido, mientras que las otras partículas son círculos sin contorno. Todo esto permite diferenciar visualmente unas partículas de otras sin gran coste computacional, asemejándolas mucho a la visión de los fuegos artificiales real.

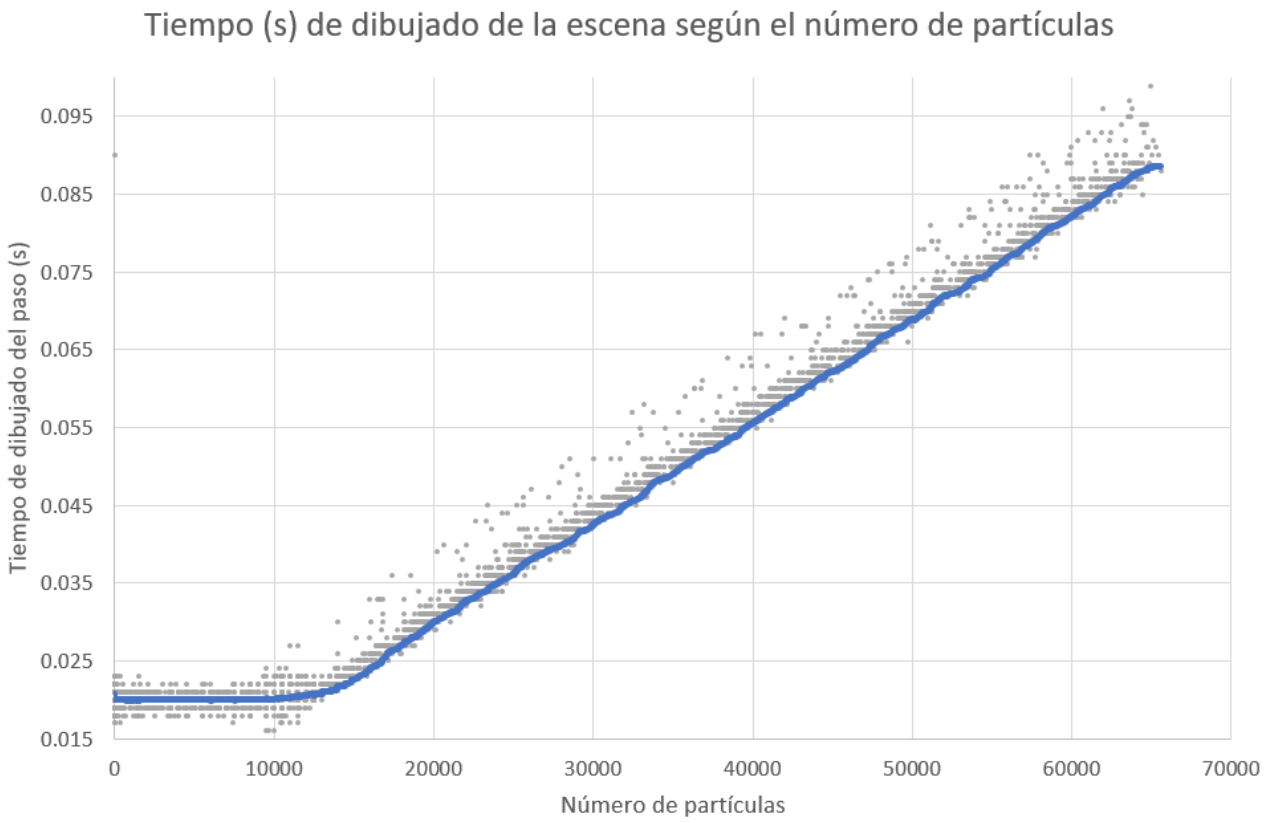
## Estudio del framerate según el número de partículas

Para realizar los estudios pertinentes nos vamos a centrar en dos valores que son calculados en cada iteración de dibujado: _deltaTimeDraw (que indica el tiempo entre cada llamada a la función de dibujado) y el frameRate de la aplicación (este se podría calcular dividiendo entre 1 los valores del deltaTimeDraw como ya se realiza en el programa base, pero mejor recurriremos a la variable predefinida en Processing 3 “frameRate” para obtener resultados algo más diferenciados).

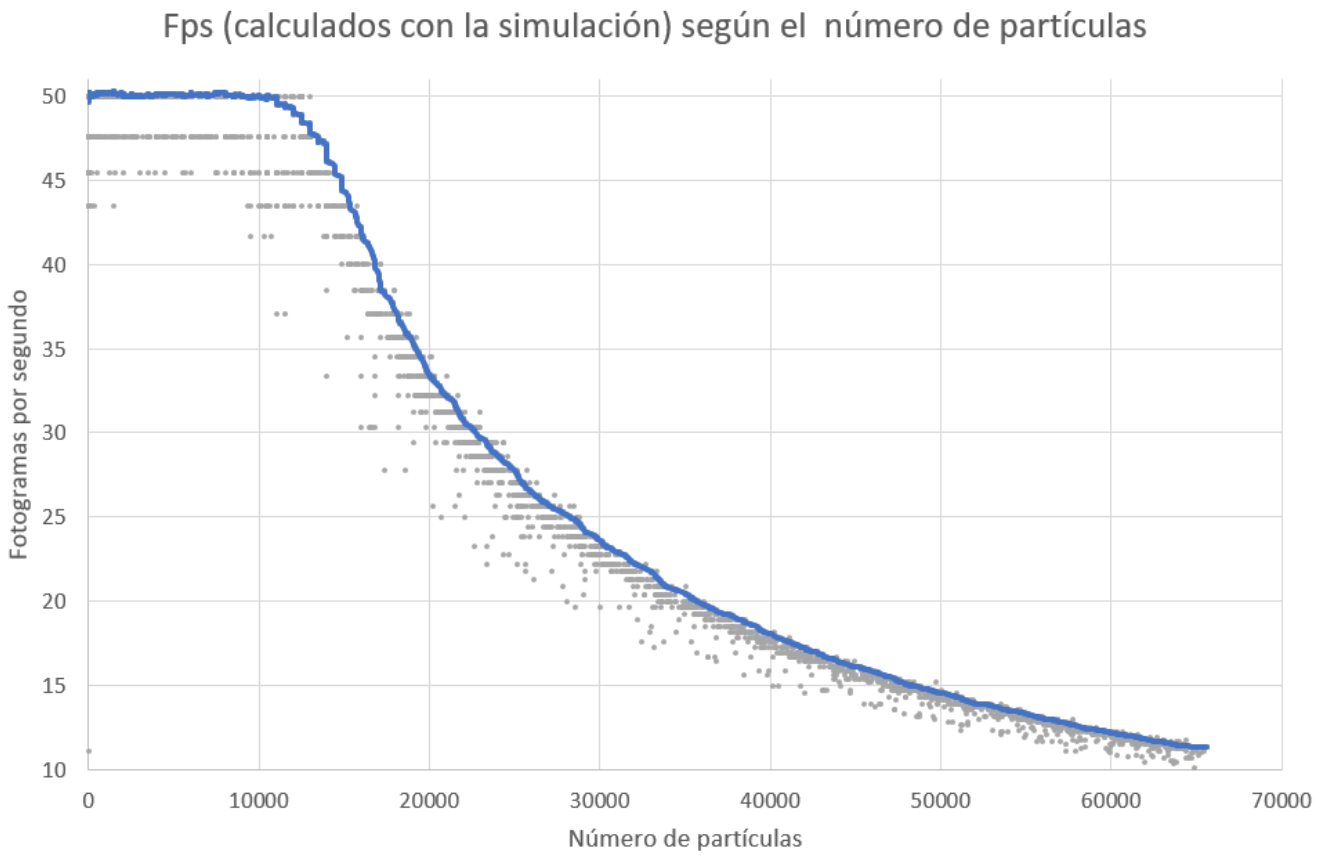
Se especifica en el documento de la práctica que para obtener resultados precisos lo adecuado es promediar y realizar varios cálculos. En nuestro caso, el proceso será algo diferente:

1. Guardaremos en un objecto de la clase Table, en cada iteración, una fila con el número de partículas, el frameRate en ese mismo instante y el tiempo de dibujado.
2. La información la guardaremos en un documento xls que abriremos con un editor externo de Hojas de Cálculo.
3. En este editor, volcaremos toda la información en una gráfica de dispersión en cuya zona de mayor densidad se definirá una línea de tendencia.
4. Aprovecharemos para añadir una tercera columna que hará el mismo cálculo que nuestro programa a partir de _deltaTimeDraw para calcular una tercera columna de frameRate según nuestra simulación.
5. Se comparará la línea de tendencia con una dada por una función de crecimiento lineal para obtener conclusiones.

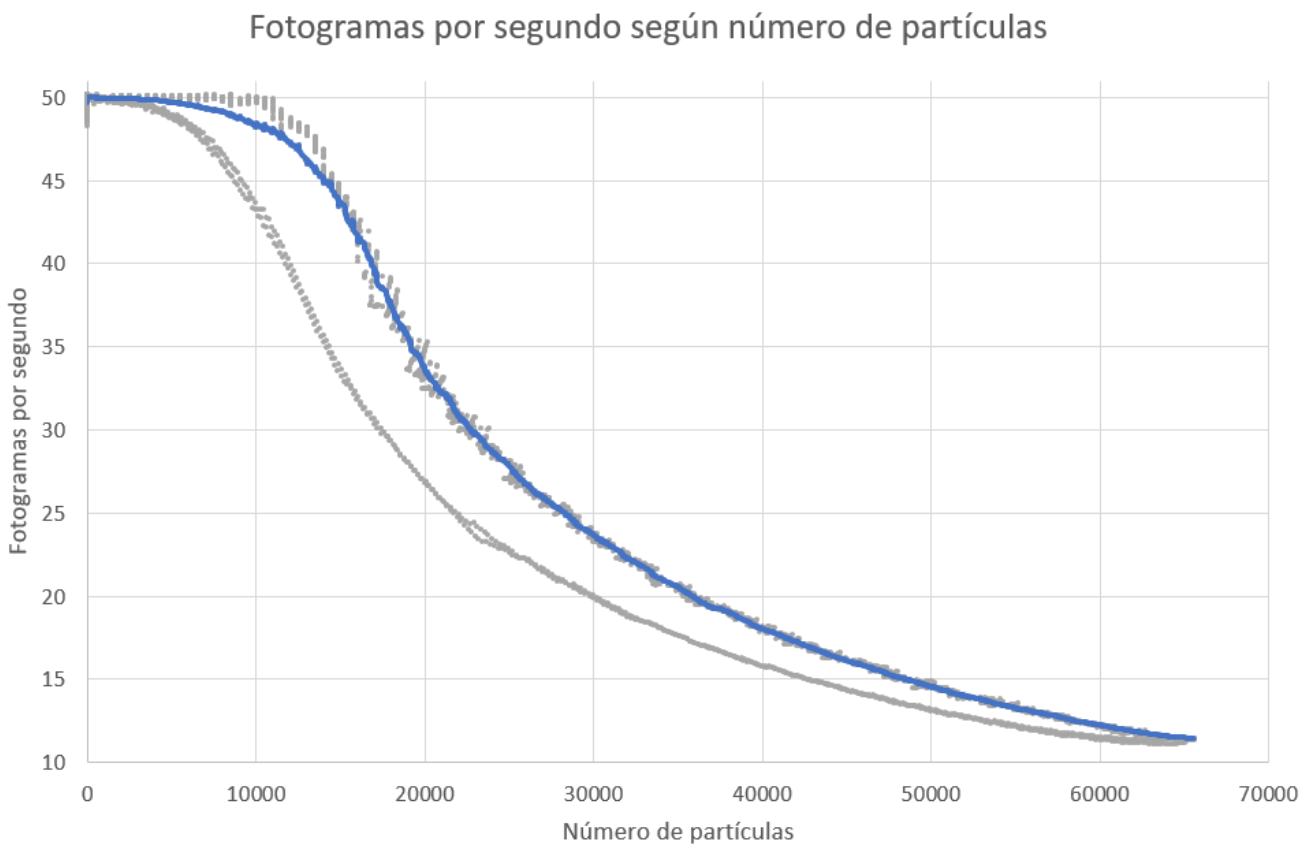
Se han realizado varias pruebas tratando de generar entre 60000 y 70000 partículas progresivamente que, debido a los tiempos de vida aleatorios, también se eliminarán progresivamente dándonos líneas de tendencia bastante claras. Sus resultados son los siguientes:




Por algún motivo, el tiempo de dibujado es prácticamente constante hasta pasar las 10000 partículas, y a partir de ahí parece adoptar un coste de O(n) (lineal respecto al número de partículas)

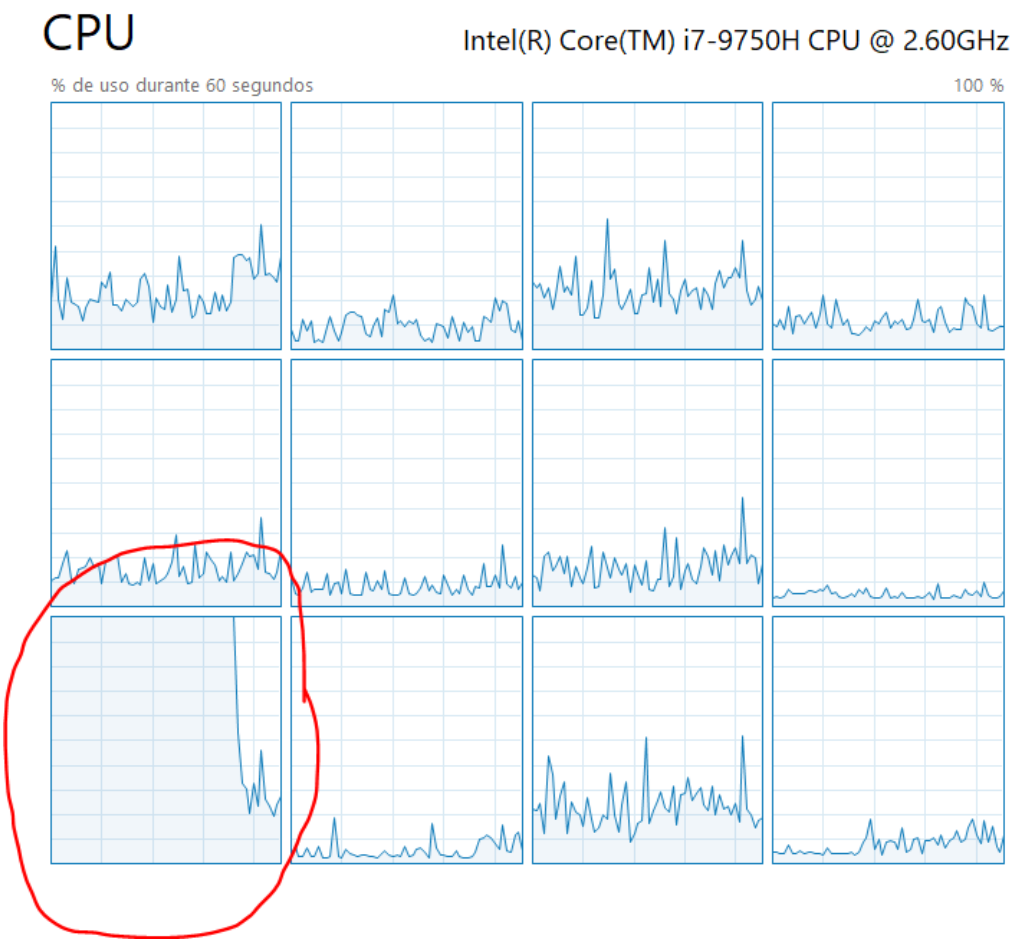


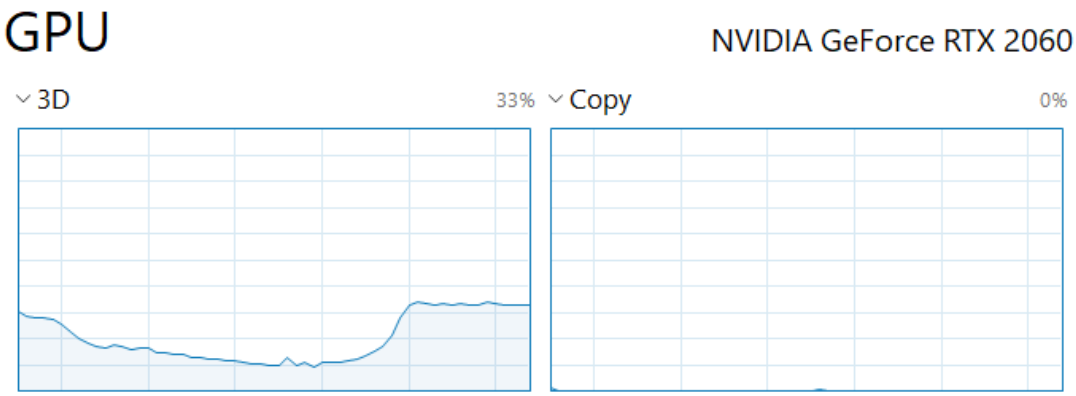
La misma situación se refleja en la gráfica de frame-rate calculado a partir del dato anterior, dándonos lugar a una proporcionalidad inversa, que implica el mismo coste lineal.




En el cálculo del frame-rate por parte de la aplicación (por lo que se documenta, algo más preciso que el nuestro) se nos muestra información adicional; y esta es la diferencia entre el coste temporal que provoca el mero dibujado de las partículas ya generadas (seguido por la línea de tendencia azul), y el coste temporal añadido que provoca dibujar las partículas mientras al mismo tiempo se resuelven los cálculos de la función explosión(). Sin embargo, ambos mantienen el mismo orden.

Lo que nos podemos preguntar, es el por qué se produce este resultado, que parece tener su origen en algún cuello de botella visible cuando se llegan a las 10000 partículas.




Ante la reducción que se da durante la creación de partículas de la ocupación de la GPU y la saturación de ese núcleo, se puede entender que nuestra simulación no está paralelizando los cálculos en diferentes núcleos, generando un cuello de botella en la CPU.

Se puede entender, por tanto, que al ser el cálculo de cada partícula independiente del resto, el coste temporal de dibujado de las partículas será constante mientras no se sature la CPU. A partir de ahí crecerá linealmente. Un efecto similar debería darse en caso de que comenzasen a llenarse los núcleos de la GPU, pues la memoria necesaria para el dibujado de la escena 2D no debería ser relevante para una tarjeta dedicada de este tipo.